

LAB PROGRAM 8: INTERFACE IMPLEMENTATION

Employee Payroll Processing System using Java Interfaces & Polymorphism

Course: Object Oriented Programming (Java)
Document Type: Technical Report & Execution Manual

Topic: Interfaces, Abstract Methods & Polymorphism
Status: Verified & Compiled

1. OBJECTIVE

The objective of this program is to design and implement a payroll calculation system in Java using **Interfaces**. The program demonstrates key Object-Oriented Programming (OOP) principles, including abstraction, dynamic method dispatch (polymorphism), and interface implementation. By defining a common Payable interface with fixed financial allowances and an abstract calculation method, the system seamlessly computes total remuneration for different categories of employees (**Regular** and **Contract**) based on their unique identification prefixes.

2. SYSTEM REQUIREMENTS

Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** 200 MB free disk space for JDK & IDE
- **Architecture:** x86 / x64 / ARM64 compatible

Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **JDK Version:** JDK 8.0 or higher (JDK 17/21 LTS recommended)
- **Text Editor / IDE:** VS Code, Eclipse, IntelliJ IDEA, or Notepad++
- **Command Shell:** CMD, PowerShell, or Bash Terminal

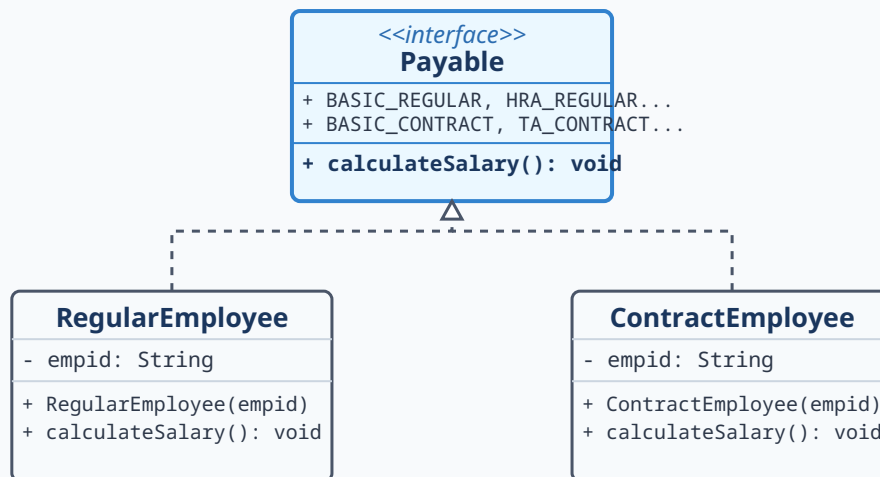
3. ALGORITHM

- 1 **Define Interface Payable:**
 - 2 ◦ Declare financial constants: BASIC_REGULAR (25000), HRA_REGULAR (15000), TA_REGULAR (5000), BASIC_CONTRACT (12000), and TA_CONTRACT (3000).
 - 3 ◦ Declare the abstract method calculateSalary().
- 4 **Implement Class RegularEmployee:**
 - 5 ◦ Implement Payable. Maintain a private field empId initialized via constructor.
 - 6 ◦ Override calculateSalary(): Sum BASIC_REGULAR, HRA_REGULAR, and TA_REGULAR. Print breakdown and total.
- 7 **Implement Class ContractEmployee:**
 - 8 ◦ Implement Payable. Maintain a private field empId initialized via constructor.
 - 9 ◦ Override calculateSalary(): Sum BASIC_CONTRACT and TA_CONTRACT. Print breakdown and total.

10 Main Execution Flow (LabProgram8):

- 11◦ Initialize Scanner object for console input.
- 12◦ Prompt user to input Employee ID (empid).
- 13◦ Convert empid to uppercase and check prefix:
 - 14▪ If it starts with 'R': Instantiate RegularEmployee assigned to interface reference variable employee.
 - 15▪ If it starts with 'C': Instantiate ContractEmployee assigned to interface reference variable employee.
 - 16▪ Otherwise: Display "Invalid Employee Id.", close scanner, and terminate.
- 17◦ Invoke employee.calculateSalary() polymorphically.
- 18◦ Close the scanner object.

4. UML CLASS DIAGRAM



5. JAVA SOURCE CODE

```
import java.util.Scanner;

// Interface created with initialized constants
interface Payable {
    double BASIC_REGULAR = 25000;
    double HRA_REGULAR = 15000;
    double TA_REGULAR = 5000;

    double BASIC_CONTRACT = 12000;
    double TA_CONTRACT = 3000;

    void calculateSalary(); // to be implemented by each employee type
}

// Regular Employee implements Payable interface
class RegularEmployee implements Payable {
    private String empid;
```

```

RegularEmployee(String empid) {
    this.empid = empid;
}

@Override
public void calculateSalary() {
    double total = BASIC_REGULAR + HRA_REGULAR + TA_REGULAR;
    System.out.println("Salary Details:");
    System.out.println("Basic Pay: " + (int) BASIC_REGULAR +
        " HRA: " + (int) HRA_REGULAR +
        " T.A: " + (int) TA_REGULAR +
        " Total Amount: " + (int) total);
}
}

// Contract Employee implements Payable interface
class ContractEmployee implements Payable {
    private String empid;

    ContractEmployee(String empid) {
        this.empid = empid;
    }

    @Override
    public void calculateSalary() {
        double total = BASIC_CONTRACT + TA_CONTRACT;
        System.out.println("Salary Details:");
        System.out.println("Basic Pay: " + (int) BASIC_CONTRACT +
            " T.A: " + (int) TA_CONTRACT +
            " Total Amount: " + (int) total);
    }
}

public class LabProgram8 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Employee Id: ");
        String empid = sc.next();

        // Interface reference used for dynamic polymorphism
        Payable employee;

        if (empid.toUpperCase().startsWith("R")) {
            employee = new RegularEmployee(empid);
        } else if (empid.toUpperCase().startsWith("C")) {
            employee = new ContractEmployee(empid);
        } else {
            System.out.println("Invalid Employee Id.");
            sc.close();
            return;
        }

        employee.calculateSalary();
        sc.close();
    }
}

```

6. COMPILATION & EXECUTION STEPS

- 1 **Save Source File:** Save the source code in a file named exactly `LabProgram8.java` (matching the public class name).
- 2 **Open Terminal/Command Prompt:** Navigate to the folder containing `LabProgram8.java` using command:
`cd path/to/your/folder`
- 3 **Compile the Program:** Run the Java Compiler (`javac`):
`javac LabProgram8.java`
Note: This generates `LabProgram8.class`, `Payable.class`, `RegularEmployee.class`, and `ContractEmployee.class` bytecodes.
- 4 **Execute the Program:** Run the Java Virtual Machine (`java`):
`java LabProgram8`

7. EXECUTION RESULTS & SAMPLE OUTPUTS

Case 1: Regular Employee Input (ID starting with 'R')

```
$ java LabProgram8 Enter Employee Id: R101 Salary Details: Basic Pay: 25000 HRA: 15000 T.A: 5000  
Total Amount: 45000
```

Case 2: Contract Employee Input (ID starting with 'C')

```
$ java LabProgram8 Enter Employee Id: C502 Salary Details: Basic Pay: 12000 T.A: 3000 Total  
Amount: 15000
```

Case 3: Invalid Employee ID Input

```
$ java LabProgram8 Enter Employee Id: E901 Invalid Employee Id.
```